

Manual for `urdf_to_standard_dh.py`

Generation of a PSDM-compatible standard DH table from one serial URDF chain

Generated for the Robot System Identification workflow

July 8, 2026

Abstract

This manual documents the Python script `urdf_to_standard_dh.py`. The script parses one serial chain from a URDF file, constructs a candidate standard Denavit-Hartenberg (DH) representation, writes the PSDM-compatible DH matrix, and verifies the result by forward-kinematics comparison. The manual describes the command-line interface, the expected YAML configuration, the generated outputs, the mathematical construction implemented by the script, and its limitations. The script intentionally keeps actuator-side encoder conversion, gearbox ratios, ball-screw conversion, and current-to-effort conversion outside the kinematic DH generation step.

Contents

1	Purpose and scope	3
1.1	Implemented behavior	3
1.2	Deliberate exclusions	3
2	Input files and command line	3
2.1	Command line	3
2.2	Dependencies	4
2.3	Configuration file	4
3	Generated outputs	4
4	Coordinate conventions	5
4.1	URDF transform convention	5
4.2	Revolute joint motion	5
4.3	Prismatic joint motion	5
4.4	Active joint axis in the URDF base frame	5
5	Standard DH model used by PSDM	6
6	Mathematical construction of DH frames	6
6.1	Line representation of joint axes	6
6.2	Closest points between consecutive joint axes	7
6.3	Choice of x axis	7
6.4	DH frame matrix	8
7	Extraction of standard DH parameters	8
8	Residual transforms	8
9	RPY normalization	9

10 Forward-kinematics validation	9
11 Sample output DH table	9
12 Using the generated table in MATLAB PSDM	10
13 Implementation map	10
14 Known limitations and acceptance criteria	10
14.1 Limitations	10
14.2 Acceptance criteria	11
15 Troubleshooting	11
16 References	12
A Minimal execution checklist	12
B Full standard DH transform for implementation checks	12

1 Purpose and scope

The script generates a standard DH table compatible with the PSDM MATLAB package. The generated numeric table has one row per active joint and six columns:

$$\text{DH}_i = \begin{bmatrix} a_i & \alpha_i & d_i & \theta_i & t_i & s_i \end{bmatrix}. \quad (1)$$

Here $t_i = 0$ denotes a revolute joint, $t_i = 1$ denotes a prismatic joint, and s_i is the PSDM joint sign. Under the implemented policy, the PSDM coordinate is the URDF joint coordinate:

$$q_{\text{PSDM},i} = q_{\text{URDF},i}. \quad (2)$$

Therefore the generated sign column is $s_i = +1$ for all active joints. The sign and scale from motor encoder counts to physical joint coordinates remain separate preprocessing data and are not embedded in the DH table.

1.1 Implemented behavior

The script performs the following operations:

1. Reads the YAML configuration.
2. Parses the URDF XML joint tree.
3. Extracts the parent chain from the configured base link to the configured tool link.
4. Folds fixed joints into the geometric transform chain.
5. Treats continuous joints as revolute joints.
6. Builds DH frames from the active joint axes at $q = 0$.
7. Extracts standard DH parameters from the relative frame transforms.
8. Writes CSV, optional MAT, JSON, Markdown reports, and a convention-sheet draft.
9. Validates the result by comparing URDF forward kinematics and DH forward kinematics on random joint samples.

1.2 Deliberate exclusions

The script does not solve the complete system-identification data-conversion problem. It does not process motor encoder counts, gear ratios, ball-screw leads, drive signs, current signs, or current-to-effort constants. Those quantities belong to the actuator-to-joint preprocessing stage and to the torque or generalized-effort measurement model.

The script also does not attempt to support every possible URDF topology. It is intentionally limited to one selected serial chain. Mimic joints are rejected. Non-serial branches outside the selected chain are ignored.

2 Input files and command line

2.1 Command line

The normal invocation is:

```
python3 urdf_to_standard_dh.py \
--config urdfToDhTemplate.yaml \
--output-dir urdf_to_dh_output
```

The URDF file can be overridden from the command line:

```
python3 urdf_to_standard_dh.py \  
  --config urdfToDhTemplate.yaml \  
  --urdf another_robot.urdf \  
  --output-dir urdf_to_dh_output
```

2.2 Dependencies

The required dependencies are:

```
python3 -m pip install numpy pyyaml
```

The optional MAT-file output requires SciPy:

```
python3 -m pip install scipy
```

The script uses only standard Python XML parsing plus NumPy. It does not require a ROS installation.

2.3 Configuration file

The relevant configuration fields are:

```
urdf:  
  upload: "urdf_eb1.1.xml"  
  base_link: "starman_arm_link_cart"  
  tool_link: "starman_arm_ids"  
  
coordinate_policy:  
  dh_q_reference: "URDF joint coordinate at q = 0"  
  joint_sign_policy: "URDF_coordinate"  
  emit_residual_transforms: true  
  
fk_validation:  
  random_samples: 100  
  position_tolerance_m: 0.001  
  orientation_tolerance_deg: 0.1  
  continuous_joint_sampling_range_rad: [-3.141592653589793, 3.141592653589793]
```

Only the two coordinate-policy values shown above are currently implemented. This restriction is intentional. Encoder offsets and drive signs should not be mixed into the purely geometric URDF-to-DH conversion.

3 Generated outputs

For the configured sample, the script writes the following files into the output directory.

File	Purpose
dh_table.csv	Human-readable PSDM DH table, including joint names and columns $[a, \alpha, d, \theta, t, s]$.
dh_table.mat	MATLAB file containing variable DH and the joint-name list. Written only if SciPy is installed and enabled.
urdf_kinematics.json	Joint order, generated DH matrix, active joint axes at $q = 0$, and residual transforms.

<code>dh_mapping_report.md</code>	Markdown report containing the generated DH table, residual transforms, and RPY normalization notes.
<code>conventions_sheet.draft.md</code>	Draft joint-convention table, including local and base-frame positive joint axes at $q = 0$.
<code>fk_validation_report.md</code>	Random-sample FK validation status and maximum errors.

4 Coordinate conventions

4.1 URDF transform convention

For each URDF joint, the script uses the joint origin transform first, followed by joint motion. For a chain of joints $j = 1, \dots, m$, the URDF forward kinematics are:

$$\mathbf{T}_{\text{URDF}}(q) = \prod_{j=1}^m \mathbf{T}_{\text{origin},j} \mathbf{T}_{\text{motion},j}(q_j), \quad (3)$$

where fixed joints have $\mathbf{T}_{\text{motion},j} = I$ and active joints contribute either a revolute or prismatic motion.

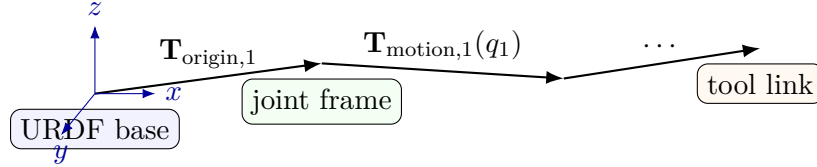


Figure 1: URDF chain evaluation used by the script. The joint origin transform is applied before the joint motion. Fixed joints are included as origin transforms only.

4.2 Revolute joint motion

For revolute and continuous joints, the joint axis is normalized and the motion transform is a rotation about that local axis. Given a unit axis $\mathbf{u}$ and joint coordinate q , the rotation matrix is computed by the Rodrigues formula:

$$R(\mathbf{u}, q) = I \cos q + (1 - \cos q) \mathbf{u} \mathbf{u}^T + [\mathbf{u}]_{\times} \sin q. \quad (4)$$

4.3 Prismatic joint motion

For prismatic joints, the motion is a translation along the normalized local axis:

$$\mathbf{T}_{\text{motion}}(q) = \begin{bmatrix} I_3 & \mathbf{u}q \\ 0 & 1 \end{bmatrix}. \quad (5)$$

4.4 Active joint axis in the URDF base frame

At $q = 0$, immediately after applying the joint origin transform, the script records:

$$\mathbf{p}_i = \text{joint origin in the URDF base frame}, \quad \mathbf{z}_{i-1} = R_{\text{base},i} \mathbf{u}_i. \quad (6)$$

The vector $\mathbf{z}_{i-1}$ becomes the DH z axis for joint i .

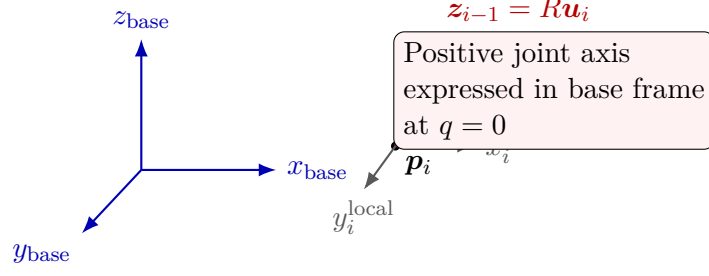


Figure 2: Transformation of a local URDF joint axis into the URDF base frame at the reference pose. For a revolute joint, positive q follows the right-hand rule about this axis. For a prismatic joint, positive q translates along this axis.

5 Standard DH model used by PSDM

The script uses the standard DH transform:

$${}^{i-1}\mathbf{T}_i = \text{Rot}_z(\theta_i) \text{Trans}_z(d_i) \text{Trans}_x(a_i) \text{Rot}_x(\alpha_i). \quad (7)$$

Expanded as a homogeneous transform:

$${}^{i-1}\mathbf{T}_i = \begin{bmatrix} \cos \theta_i & -\sin \theta_i \cos \alpha_i & \sin \theta_i \sin \alpha_i & a_i \cos \theta_i \\ \sin \theta_i & \cos \theta_i \cos \alpha_i & -\cos \theta_i \sin \alpha_i & a_i \sin \theta_i \\ 0 & \sin \alpha_i & \cos \alpha_i & d_i \\ 0 & 0 & 0 & 1 \end{bmatrix}. \quad (8)$$

PSDM then applies the active joint coordinate through t_i and s_i :

$$d_i^* = d_i + t_i s_i q_i, \quad (9)$$

$$\theta_i^* = \theta_i + (1 - t_i) s_i q_i. \quad (10)$$

In this script, $s_i = +1$ because q_i is the URDF joint coordinate and the DH frames are built to use that coordinate direction.

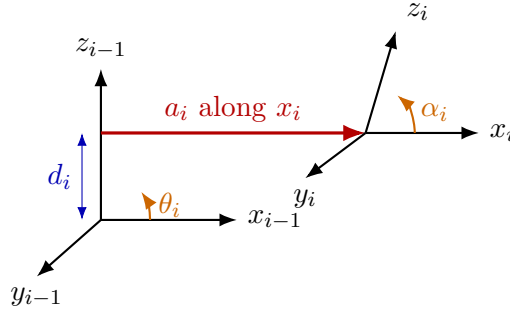


Figure 3: Geometric meaning of the standard DH parameters. The implementation extracts a_i , α_i , d_i , and θ_i from the relative transform between consecutive constructed DH frames.

6 Mathematical construction of DH frames

6.1 Line representation of joint axes

At the URDF reference pose, each active joint axis is represented as a line:

$$L_i(s) = \mathbf{p}_i + s \mathbf{z}_{i-1}, \quad s \in \mathbb{R}. \quad (11)$$

The point $\mathbf{p}_i$ lies on the joint axis and $\mathbf{z}_{i-1}$ is the positive joint axis expressed in the URDF base frame.

6.2 Closest points between consecutive joint axes

For two consecutive active axes, the script finds closest points on the two infinite lines:

$$L_i(s) = \mathbf{p}_i + s\mathbf{u}, \quad (12)$$

$$L_{i+1}(t) = \mathbf{p}_{i+1} + t\mathbf{v}. \quad (13)$$

Here $\mathbf{u}$ and $\mathbf{v}$ are unit vectors. Define:

$$b = \mathbf{u}^T \mathbf{v}, \quad \mathbf{w} = \mathbf{p}_i - \mathbf{p}_{i+1}, \quad d = \mathbf{u}^T \mathbf{w}, \quad e = \mathbf{v}^T \mathbf{w}. \quad (14)$$

If the axes are not parallel, the denominator is:

$$\Delta = 1 - b^2. \quad (15)$$

The closest-line parameters are:

$$s = \frac{be - d}{\Delta}, \quad (16)$$

$$t = \frac{e - bd}{\Delta}. \quad (17)$$

The source and target closest points are:

$$\mathbf{c}_i = \mathbf{p}_i + s\mathbf{u}, \quad (18)$$

$$\mathbf{c}_{i+1} = \mathbf{p}_{i+1} + t\mathbf{v}. \quad (19)$$

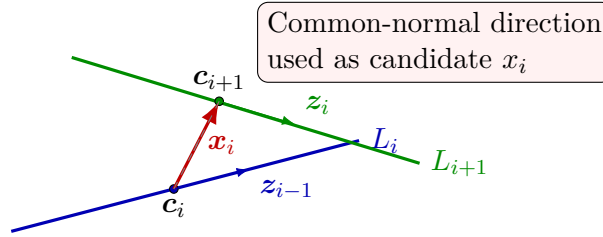


Figure 4: For skew axes, the common normal between closest points defines the DH x direction. This is the normal case used by the script for consecutive non-parallel axes.

6.3 Choice of x axis

The x_i direction is chosen according to a simple ordered rule:

1. If the closest-point separation $\mathbf{c}_{i+1} - \mathbf{c}_i$ is nonzero, set:

$$\mathbf{x}_i = \frac{\mathbf{c}_{i+1} - \mathbf{c}_i}{\|\mathbf{c}_{i+1} - \mathbf{c}_i\|}. \quad (20)$$

2. If consecutive axes intersect, use the normalized cross product when available:

$$\mathbf{x}_i = \frac{\mathbf{z}_{i-1} \times \mathbf{z}_i}{\|\mathbf{z}_{i-1} \times \mathbf{z}_i\|}. \quad (21)$$

3. If the axes are parallel or nearly parallel, reuse the previous x direction projected orthogonally to the current z axis, when possible.
4. If no previous direction is available, choose an arbitrary perpendicular basis direction.

This rule is deterministic for a given URDF and avoids adding extra dependencies. It is still a frame-construction convention, not a proof that the generated DH table is physically preferred. FK validation is the acceptance criterion.

6.4 DH frame matrix

Once $\mathbf{x}_i$ and $\mathbf{z}_i$ are chosen, the script constructs a right-handed frame:

$$\mathbf{y}_i = \frac{\mathbf{z}_i \times \mathbf{x}_i}{\|\mathbf{z}_i \times \mathbf{x}_i\|}, \quad (22)$$

$$\mathbf{x}_i \leftarrow \frac{\mathbf{y}_i \times \mathbf{z}_i}{\|\mathbf{y}_i \times \mathbf{z}_i\|}. \quad (23)$$

Then:

$$F_i = \begin{bmatrix} \mathbf{x}_i & \mathbf{y}_i & \mathbf{z}_i & \mathbf{o}_i \\ 0 & 0 & 0 & 1 \end{bmatrix}. \quad (24)$$

Here $\mathbf{o}_i$ is the selected origin for frame i .

7 Extraction of standard DH parameters

For each active joint i , the script computes the relative transform:

$${}^{i-1}\mathbf{T}_i = F_{i-1}^{-1}F_i = \begin{bmatrix} R & \mathbf{p} \\ 0 & 1 \end{bmatrix}. \quad (25)$$

The standard DH parameters are then extracted as:

$$\theta_i = \text{atan2}(R_{21}, R_{11}), \quad (26)$$

$$\alpha_i = \text{atan2}(R_{32}, R_{33}), \quad (27)$$

$$a_i = \cos \theta_i p_x + \sin \theta_i p_y, \quad (28)$$

$$d_i = p_z. \quad (29)$$

The implementation checks that the relative transform is consistent with the standard DH form by verifying that:

$$R_{31} \approx 0, \quad -\sin \theta_i p_x + \cos \theta_i p_y \approx 0. \quad (30)$$

If this check fails, the frame construction did not produce a valid standard DH relative transform and the script aborts.

8 Residual transforms

The generated DH frame chain is allowed to differ from the selected URDF base and tool link frames. The script therefore writes two residual transforms:

$$\mathbf{T}_{\text{base} \rightarrow \text{DH0}} = F_0, \quad (31)$$

$$\mathbf{T}_{\text{DHn} \rightarrow \text{tool}} = F_n^{-1} \mathbf{T}_{\text{tool}}(0). \quad (32)$$

The validation equation is:

$$\mathbf{T}_{\text{URDF}}(q) = \mathbf{T}_{\text{base} \rightarrow \text{DH0}} \left(\prod_{i=1}^n {}^{i-1}\mathbf{T}_i(q_i) \right) \mathbf{T}_{\text{DHn} \rightarrow \text{tool}}. \quad (33)$$

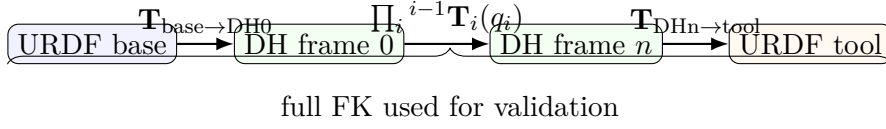


Figure 5: Residual transforms preserve the exact relationship between the selected URDF frames and the generated DH frame chain. PSDM receives the DH table; validation and documentation retain the residual transforms.

9 RPY normalization

The sample URDF uses low-precision values such as 1.57 and 3.14 for nominal rotations of $\pi/2$ and π . Direct use of those literals can create artificial near-parallel or near-intersecting axes and produce large numerical DH offsets.

The script snaps each URDF RPY component to the nearest multiple of $\pi/2$ if:

$$|r_{\text{URDF}} - k\pi/2| \leq 10^{-2} \text{ rad}, \quad k \in \mathbb{Z}. \quad (34)$$

The same normalized transforms are used for both DH construction and URDF FK validation. The mapping report lists all joints whose RPY values were normalized.

10 Forward-kinematics validation

The validation process samples joint configurations and compares:

$$\mathbf{T}_{\text{URDF}}(q) \quad \text{against} \quad \mathbf{T}_{\text{DH}}(q). \quad (35)$$

For revolute joints with finite limits, the script samples uniformly in the URDF limits. Continuous joints are sampled from the configured range. Prismatic joints are sampled in their linear limits.

The position error is:

$$e_p = \|\mathbf{p}_{\text{URDF}} - \mathbf{p}_{\text{DH}}\|_2. \quad (36)$$

The orientation error in degrees is obtained from the relative rotation:

$$e_R = \cos^{-1} \left(\frac{\text{tr}(R_{\text{URDF}}^T R_{\text{DH}}) - 1}{2} \right) \frac{180}{\pi}. \quad (37)$$

The argument of $\cos^{-1}$ is clipped to $[-1, 1]$ to avoid numerical round-off failures.

For the supplied sample, the generated validation report gave:

$$\max(e_p) = 8.03 \times 10^{-16} \text{ m}, \quad (38)$$

$$\max(e_R) = 1.71 \times 10^{-6} \text{ deg}. \quad (39)$$

11 Sample output DH table

The generated sample table for `urdf_eb1.1.xml` and `urdfToDhTemplate.yaml` is shown below.

i	Joint	a (m)	α (rad)	d (m)	θ (rad)	t	s
1	starman_arm_joint1	0.0117153745	0	0	0	1	1
2	starman_arm_joint2	0	1.5707963268	0.036	0.6947382762	0	1
3	starman_arm_joint3	0	1.5707963268	0.797289	3.1415926536	1	1
4	starman_arm_joint4	0	1.5707963268	-0.161843	1.5707963268	0	1

5	starman_arm_joint5	0.25	0	0.03	-1.5707963268	0	1
6	starman_arm_joint6	0.1199	-1.5707963268	-0.03	-1.5707963268	0	1
7	starman_arm_joint7	0	0	0	0	0	1

12 Using the generated table in MATLAB PSDM

The CSV file is intended for review and version control. The MAT file is directly loadable in MATLAB:

```
load('urdf_to_dh_output/dh_table.mat', 'DH', 'joint_names');
g = [0; 0; 1]; % adjust to the PSDM base frame convention
[E, P] = PSDM.deriveModel(DH, g);
```

The gravity vector must be expressed in the PSDM base frame. If the PSDM base frame is not identical to the URDF base link frame, use the residual transform $T_{urdf_base_to_dh_base}$ from `urdf_kinematics.json` to document and verify the intended gravity direction.

13 Implementation map

Function	Role
<code>parse_urdf</code>	Reads URDF joints, parent and child links, origin transforms, axes, limits, and mimic flags.
<code>get_chain</code>	Walks from the configured tool link to the configured base link through parent links, then reverses the chain.
<code>urdf_fk</code>	Evaluates URDF forward kinematics and records active joint axes at $q = 0$.
<code>make_dh_frames</code>	Constructs DH frames from active joint axes using closest points and common normals.
<code>dh_row</code>	Extracts a , α , d , and θ from a relative standard-DH transform.
<code>dh_fk</code>	Evaluates the generated DH chain, including residual transforms.
<code>validate_fk</code>	Performs random-sample FK comparison and writes the validation report.
<code>write_* functions</code>	Write CSV, MAT, JSON, mapping report, convention-sheet draft, and validation report.

14 Known limitations and acceptance criteria

14.1 Limitations

1. The script supports one selected serial chain only.
2. Mimic joints are not supported.
3. Only the coordinate policy $q_{PSDM} = q_{URDF}$ is implemented.
4. The generated DH frame assignment is a deterministic convention, not a unique representation of the robot.
5. RPY snapping is intended for URDFs that use approximate literals for exact right-angle rotations. It should be reviewed if the robot has intentional small-angle offsets near multiples of $\pi/2$.

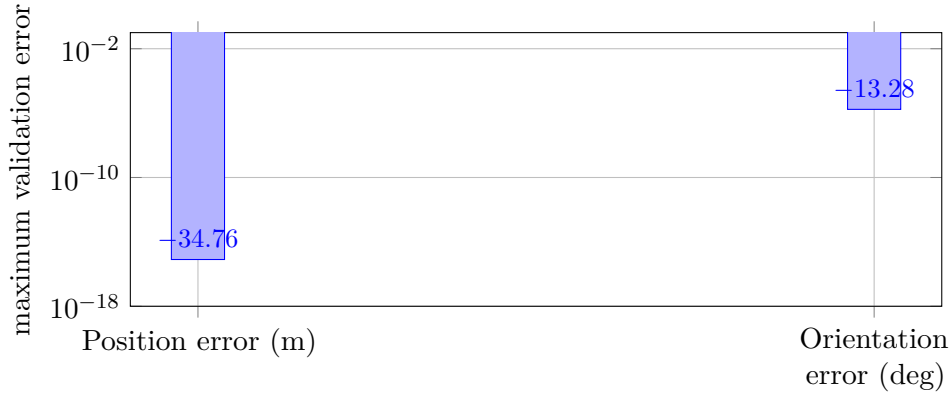


Figure 6: Maximum FK validation errors reported for the supplied sample URDF and configuration. The plot uses a logarithmic vertical axis because the position and orientation errors have different units and magnitudes.

6. The script generates kinematic data only. It does not generate dynamic parameters, actuator constants, friction parameters, or telemetry conversion constants.

14.2 Acceptance criteria

The generated DH table should be accepted only if:

1. The selected base and tool link names are correct.
2. The active joint order matches the physical and controller joint order expected by the identification workflow.
3. The convention-sheet draft is reviewed by someone familiar with the mechanism.
4. The FK validation report passes the configured position and orientation tolerances.
5. The residual transforms are stored with the model artifacts.
6. The actuator-to-joint telemetry conversion remains documented separately.

15 Troubleshooting

Symptom	Likely cause and corrective action
No parent chain exists	The configured tool link is not a link name on a descendant path from the configured base link. Check that a link name, not a joint name, is used.
Mimic joint is not supported	The selected chain contains a mimic joint. Provide an independent coordinate mapping or choose a different chain endpoint.
A selected joint has a zero-length axis	A selected active URDF joint has an invalid axis vector. Check that the chosen chain does not include nonstandard tags or malformed active joints.
Internal DH frame construction failed	The constructed consecutive frames did not satisfy the standard-DH relative transform form. Inspect the axis geometry and generated residual transforms.

Validation fails	Check joint order, active joint axes, base and tool links, continuous-joint sampling ranges, and whether RPY normalization is appropriate.
DH values are large or unstable	Check for near-parallel axes caused by low-precision URDF rotations, incorrect axis definitions, or a tool endpoint that should instead be treated as a residual fixed transform.

16 References

1. S. Lloyd, R. Irani, and M. Ahmadi, *A numeric derivation for fast regressive modeling of manipulator dynamics*, Mechanism and Machine Theory, 2021.
2. S. Lloyd, R. Irani, and M. Ahmadi, *Application of Pseudo-Symbolic Dynamic Modeling (PSDM) in the Modeling and Calibration of a 6-DOF Articulated Robot*, preprint.
3. PSDM MATLAB code package README.
4. M. W. Spong and M. Vidyasagar, *Robot Dynamics and Control*, referenced by the PSDM README for the DH convention.

A Minimal execution checklist

1. Install Python dependencies: `numpy`, `pyyaml`, and optionally `scipy`.
2. Verify YAML base and tool link names.
3. Run the script.
4. Inspect `dh_mapping_report.md` and `conventions_sheet.draft.md`.
5. Confirm FK validation status is `PASS`.
6. Commit the DH table, residual transforms, configuration, and validation report with the model-generation artifacts.

B Full standard DH transform for implementation checks

For any row $[a, \alpha, d, \theta, t, s]$, the runtime coordinate substitution is:

$$\theta^* = \theta + (1 - t)sq, \quad d^* = d + tsq. \quad (40)$$

The evaluated transform is:

$$A(q) = \begin{bmatrix} \cos \theta^* & -\sin \theta^* \cos \alpha & \sin \theta^* \sin \alpha & a \cos \theta^* \\ \sin \theta^* & \cos \theta^* \cos \alpha & -\cos \theta^* \sin \alpha & a \sin \theta^* \\ 0 & \sin \alpha & \cos \alpha & d^* \\ 0 & 0 & 0 & 1 \end{bmatrix}. \quad (41)$$